

MATH5350M: Computations in Finance

Coursework 3 (35%): Due Thursday 7th May at 14:00

Nadhir Ben Rached (n.benrached@leeds.ac.uk)
University of Leeds, 2026

Your work should be submitted on Minerva before 14:00 on Thursday 7th May. Late submissions will be penalized 5% for each day that they are late. Work that is submitted more than a week late will get zero marks.

Further details on how to submit are given below.

In this coursework, you should solve the Black-Scholes PDE for a vanilla European call option using the implicit and Crank-Nicolson methods, and numerically compute their error orders. Write your programs in python and please use the function names and arguments as suggested in the detailed descriptions below. You should write a report with the answers to the questions in Parts 2 to 6. In your report, describe how you overcame any programming challenges you faced.

1 Inputting data (10 marks)

Implement a function responsible for inputting data. This function should be informative, user friendly and must verify correctness of arguments. You are expected to deal with situations when an inputted parameter is clearly wrong: for example, when the stock price is negative or a letter, not a number. Please use the function name:

`input_data()`

The function should return the following parameters:

- `r` — risk-free interest rate r (per annum),
- `sigma` — stock price volatility σ (per annum),
- `T` — option contract length T ,
- `K` — strike price K of the option,
- `M` — number of grid intervals in the price variable,
- `N` — number of grid intervals in the time variable,
- `xmin` — the minimum x -value in the domain,
- `xmax` — the maximum x -value in the domain.

2 Thomas algorithm (15 marks)

Write a program with function name and arguments

`tridiagonal_solve_thomas(alpha,beta,gamma,b)`

that implements the Thomas algorithm (algorithm 7.3 in the notes) to solve a linear system of equations of the form

$$\begin{bmatrix} \alpha & \beta & 0 & \dots & 0 \\ \gamma & \alpha & \beta & \ddots & \vdots \\ 0 & \gamma & \alpha & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & \beta \\ 0 & \dots & 0 & \gamma & \alpha \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \\ \vdots \\ b_n \end{bmatrix},$$

where α, β, γ correspond to the inputs `alpha`, `beta`, `gamma` respectively, and the vector $(b_1, b_2, \dots, b_n)^T$ corresponds to the array `b`. Confirm that your algorithm works by comparing the output to a simple example that you can compute by hand.

Write a program with function name and arguments

```
tridiagonal_solve_banded(alpha,beta,gamma,b)
```

with the same inputs and output as `tridiagonal_solve_thomas` that uses the `solve_banded` function in the `linalg` module of `scipy`. Check that both algorithms produce the same results. Which algorithm is faster? Comment on why you think this is in your report.

3 Numerical solution of the Black-Scholes PDE — implicit scheme (20 marks)

Consider the standard Black-Scholes market model, i.e. there is one bond, priced at $B_t = e^{rt}$, and one stock, whose price process under the risk neutral measure satisfies

$$dS_t = rS_t dt + \sigma S_t dW_t,$$

where W_t is a Brownian motion, r is an interest rate and σ is a volatility. The price of an option on this stock satisfies the following partial differential equation:

$$\frac{1}{2}\sigma^2 s^2 \frac{\partial^2 V(s,t)}{\partial s^2} + rs \frac{\partial V(s,t)}{\partial s} - rV(s,t) + \frac{\partial V(s,t)}{\partial t} = 0.$$

In your report, write down the final-time and boundary conditions which accompany this PDE for a vanilla European Call option in both the untransformed (s, t) and transformed (x, τ) variables.

Write a program with function name and arguments

```
black_scholes_implicit(r,sigma,T,K,M,N,xmin,xmax)
```

which solves the heat equation using the implicit scheme for the corresponding vanilla European Call option with $r = 0.05$, $\sigma = 0.1$, $T = 1.8$ and $K = 673$. Your program should return the present value of the option and the corresponding initial stock prices. Describe how you chose the values of $x_{\min}$ and $x_{\max}$, and what values the numerical solution takes at these points. Check that your numerical solution is correct by comparing it to the exact solution computed using formula 1.19 in the notes.

4 Numerical solution of the Black-Scholes PDE — Crank-Nicolson scheme (20 marks)

Repeat Part 3 but now using the Crank-Nicolson scheme, using the function name and arguments

```
black_scholes_crank_nicolson(r,sigma,T,K,M,N,xmin,xmax).
```

5 Comparison of errors (20 marks)

It is stated in the notes that the implicit scheme has error of order $\mathcal{O}(\delta\tau) + \mathcal{O}((\delta x)^2)$ and the Crank-Nicolson scheme has error of order $\mathcal{O}((\delta\tau)^2) +$

$\mathcal{O}((\delta x)^2)$. Use your programs to numerically compare the error orders of the implicit and Crank-Nicolson schemes and check whether they agree with the theoretical error orders. Describe what error metric you use and why.

6 Results and discussion (15 marks)

Summarise your findings and discuss your results.

7 Directions for submission

You need to submit a report and the code.

Report: Your report must be typed in Microsoft Word or Latex. Do not include the code in your report. Structure your report as the assignment (Parts 2 to 6), leaving out Part 1 which does not require an answer. Attach the **Academic Integrity Form** to your report.

Your report (as a PDF file) must be uploaded in the Submit My Work area using the TurnItIn submission tool entitled “*Report submission for Coursework 3*”.

Code: Before submission, clean up the code and check that the programme runs. Make sure you have not shared your code or a piece of thereof with anyone else as this qualifies as plagiarism.

The python file `cw3-studentnumber.py` (replace `studentnumber` with your student number; NO pdf, doc, ps, etc.) must be uploaded via “*Program submission for Coursework 3*” in Gradescope.

Marking: The following aspects of your report are important in the marking: quality of the answers, presentation (easy to understand, looking professional, quality and labelling of figures), relevance, conciseness (excluding any figures, you can do good work in two pages of text, five is probably more than necessary; think carefully about what figures you include and how you present your results).

The following aspects of your programme are important in the marking: correctness (the programme does what it is supposed to do), quality of code (e.g., meaningful names for variables, comments that facilitate understanding of the code) and efficiency (speed and memory usage).

Deadline: Both the report and the code have to be submitted by the deadline. Please ensure that you leave sufficient time to complete the online submission process, as upload times can vary. It is your responsibility to ensure you upload the correct files to Minerva, and that it has uploaded successfully.

8 Python code suggestions

- The `numpy` function `tile(A, (m,n))` repeats the array `A`, `m` times vertically and `n` times horizontally.
- The `numpy` function `concatenate((a1,a2,...))` joins the sequence of arrays `(a1,a2,...)`.
- The code

```
numpy.round(numpy.logspace(a1,a2,n)).astype(int)
```

produces an array of `n` integers between `10**a1` and `10**a2` that are evenly spaced in log-coordinates.